

Tugas Mandiri Modul 7

Nama : Rovino Ramadhani

NIM : 103072400031

Kelas : IF 04-01

Penjelasan Soal 1

Code

<https://github.com/rovinor4/Praktikum-Dasar-Kecerdasan-Artifisial/tree/main/Tugas%20Mandiri%20-%20Modul%207>

```
In [5]: import heapq
```

Kode ini digunakan untuk mengimpor library heapq yang berfungsi sebagai priority queue. Pada algoritma UCS, priority queue dipakai untuk mengambil node dengan cost terkecil terlebih dahulu.

```
In [6]: graph_soal_1 = {
    "A": [("B", 4), ("C", 2)],
    "B": [("A", 4), ("D", 5), ("E", 10)],
    "C": [("A", 2), ("D", 8), ("F", 3)],
    "D": [("B", 5), ("C", 8), ("E", 2), ("F", 6)],
    "E": [("B", 10), ("D", 2), ("G", 3)],
    "F": [("C", 3), ("D", 6), ("G", 7)],
    "G": [("E", 3), ("F", 7)]
}
```

Mendefinisikan graf untuk soal pertama dalam format dictionary. Key merupakan nama simpul (node), sedangkan value merupakan daftar tupel yang berisi simpul tetangga beserta biaya (jarak) menuju simpul tersebut.

```
In [7]: def ucs_soal_1(graph, start, goal):
    queue = [(0, start, [start])]
    visited = set()

    while queue:
        cost, node, path = heapq.heappop(queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
```

```

        return path, cost

    for neighbor, edge_cost in graph.get(node, []):
        if neighbor not in visited:
            heapq.heappush(queue, (cost + edge_cost, neighbor, 1))

    return None, None

```

Fungsi implementasi algoritma Uniform Cost Search (UCS). Fungsi ini menggunakan heapq untuk selalu mengevaluasi jalur dengan total biaya terkecil lebih dahulu. Variabel visited digunakan agar tidak mengevaluasi node yang sama berulang kali. Jika node tujuan (goal) tercapai, fungsi akan mengembalikan jalur (path) dan total biaya (cost).

```

In [8]: start = input("Masukkan node awal: ").upper()
        goal = input("Masukkan node tujuan: ").upper()

```

Mengeksekusi fungsi ucs_soal_1 berdasarkan graf pertama dan input pengguna.

```

In [9]: path, cost = ucs_soal_1(graph_soal_1, start, goal)

```

```

In [10]: if path:
          print("Jalur terpendek:", " -> ".join(path))
          print("Total cost:", cost)
        else:
          print("Jalur tidak ditemukan")

```

Jalur terpendek: A -> C -> F -> G
Total cost: 12

Mengecek keberhasilan pencarian rute. Jika rute ada (path tidak kosong), program mencetak jalur menggunakan pemisah " -> " dan menampilkan biayanya.

Penjelasan Soal 2

```

In [11]: graph_soal_2 = {
          "Jakarta": [("Bandung", 120), ("Cirebon", 200)],
          "Bandung": [("Jakarta", 120), ("Semarang", 350), ("Yogyakarta", 400)],
          "Cirebon": [("Jakarta", 200), ("Semarang", 250)],
          "Semarang": [("Bandung", 350), ("Cirebon", 250), ("Surabaya", 300)],
          "Yogyakarta": [("Bandung", 400), ("Surabaya", 330)],
          "Surabaya": [("Semarang", 300), ("Yogyakarta", 330)]
        }

```

Mendefinisikan graf untuk soal kedua berupa peta kota beserta jaraknya (biaya). Format penyimpanan menggunakan struktur dictionary yang sama dengan soal pertama.

```
In [12]: def tampilkan_graph(graph):
          print("Daftar graph:")
          for node, neighbors in graph.items():
              print(node, ":", neighbors)
```

Fungsi ini digunakan semata-mata untuk mencetak struktur dictionary graf ke layar agar pengguna dapat melihat seluruh isi simpul dan sisi/jaraknya.

```
In [13]: def ucs_soal_2(graph, start, goal):
          queue = [(0, start, [start])]
          visited = set()
          visited_order = []
          expansion_order = []

          while queue:
              cost, node, path = heapq.heappop(queue)

              if node in visited:
                  continue

              visited.add(node)
              visited_order.append(node)
              expansion_order.append(node)

              if node == goal:
                  return path, cost, visited_order, expansion_order

              for neighbor, edge_cost in graph.get(node, []):
                  if neighbor not in visited:
                      heapq.heappush(queue, (cost + edge_cost, neighbor, path + [neighbor]))

          return None, None, visited_order, expansion_order
```

Modifikasi fungsi pencarian (UCS) untuk soal kedua. Fungsi ini menambahkan fitur pelacakan (visited_order dan expansion_order) untuk mencatat urutan node ketika dikunjungi dan diekspansi secara berurutan. Semua parameter ini kemudian dikembalikan (di-return) di akhir pencarian.

```
In [14]: tampilkan_graph(graph_soal_2)

start = input("Masukkan kota asal: ").title()
goal = input("Masukkan kota tujuan: ").title()
```

Daftar graph:

```
Jakarta : [('Bandung', 120), ('Cirebon', 200)]
Bandung : [('Jakarta', 120), ('Semarang', 350), ('Yogyakarta', 400)]
Cirebon : [('Jakarta', 200), ('Semarang', 250)]
Semarang : [('Bandung', 350), ('Cirebon', 250), ('Surabaya', 300)]
Yogyakarta : [('Bandung', 400), ('Surabaya', 330)]
Surabaya : [('Semarang', 300), ('Yogyakarta', 330)]
```

Memanggil fungsi untuk menampilkan daftar kota dari graph_soal_2, kemudian meminta masukan pengguna. Fungsi .title() digunakan untuk

mengubah format teks sehingga huruf pertama dari setiap kata menjadi kapital (contoh: "jakarta" menjadi "Jakarta").

```
In [15]: path, cost, visited_order, expansion_order = ucs_soal_2(graph_soal_2,
```

Mengeksekusi fungsi pencarian pada graf kedua menggunakan input asal dan tujuan.

```
In [16]: if path:
    print("Jalur terbaik:", " -> ".join(path))
    print("Total biaya:", cost)
    print("Urutan node yang dikunjungi:", " -> ".join(visited_order))
    print("Urutan ekspansi node:", " -> ".join(expansion_order))
else:
    print("Jalur tidak ditemukan")
    print("Urutan node yang dikunjungi:", " -> ".join(visited_order))
    print("Urutan ekspansi node:", " -> ".join(expansion_order))
```

Jalur terbaik: Jakarta -> Cirebon -> Semarang -> Surabaya

Total biaya: 750

Urutan node yang dikunjungi: Jakarta -> Bandung -> Cirebon -> Semarang -> Yogyakarta -> Surabaya

Urutan ekspansi node: Jakarta -> Bandung -> Cirebon -> Semarang -> Yogyakarta -> Surabaya

Mengecek keberhasilan algoritma, kemudian mencetak keseluruhan parameter pencarian yang dikembalikan dari ucs_soal_2 (termasuk path, cost, urutan visited, dan urutan expansion).